

AI Smart Device Control & Automation Assistant

STEP 03 PROJECT DOCUMENTATION — Command Parser / Intent Model

Item	Current Project Details
Completed Steps	01 — Initialization; 02 — Configuration & Command Model; 03 — Command Parser / Intent Model
Python	Python 3.11.9 — venv
Test Framework	pytest 9.1.1
Parser	src/command_parser.py
Automated Tests	5/5 PASS — 100%
Step Status	COMPLETED — PASS

1. Executive Summary

Step 03 implemented a deterministic local command parser that converts simple natural-language commands into the structured **DeviceCommand** model created in Step 02. The parser normalizes input, recognizes initial IoT, laptop, and Bluetooth intents, and safely represents unsupported input as an unknown command. Five dedicated pytest tests were executed and all five passed in 0.06 seconds.

2. Command Processing Architecture

User Text ↓ `normalize_text()` ↓ `parse_command()` ↓ `DeviceCommand` ↓ Future Device Controller
Example: "Turn on the desk light" ↓ `device = iot` `action = turn_on` `target = desk_light`

The parser only produces structured commands at this stage. It does not execute laptop, Bluetooth, or IoT actions.

3. Command Parser Implementation

The parser uses deterministic rules for the initial supported intents. Input is normalized before matching.

Input Example	Device	Action	Target
Turn on the desk light	iot	turn_on	desk_light
Turn off the desk light	iot	turn_off	desk_light
Turn on the fan	iot	turn_on	fan
Turn off the fan	iot	turn_off	fan
Open Chrome	laptop	open	chrome
Open VSCode	laptop	open	vscode
Show Bluetooth devices	bluetooth	list_devices	None
Do something unknown	unknown	unknown	normalized command

4. Structured Command Example

```
{ "device": "iot", "action": "turn_on", "target": "desk_light", "value": null }
```

5. Unknown Command Handling

Unsupported input is not executed. The parser returns **device = unknown** and **action = unknown**, retaining the normalized command as the target field. This provides a safe fallback.

6. Automated Testing

The final pytest run was performed from the project root using Python 3.11.9 and pytest 9.1.1.

Test	Result
test_turn_on_light	PASSED
test_turn_off_fan	PASSED
test_open_chrome	PASSED
test_bluetooth_devices	PASSED
test_unknown_command	PASSED
Total	5 passed in 0.06s

7. Final Test Result

```
pytest -v collected 5 items tests/test_command_parser.py::test_turn_on_light PASSED
tests/test_command_parser.py::test_turn_off_fan PASSED tests/test_command_parser.py::test_open_chrome
PASSED tests/test_command_parser.py::test_bluetooth_devices PASSED
tests/test_command_parser.py::test_unknown_command PASSED 5 passed in 0.06s
```

8. Python Package / Import Validation

The project package structure was corrected by adding **src/___init___.py**. Final validation confirmed that the src package and parser import worked successfully with the message **IMPORT PASS**.

```
python -c "import src; print(src.__file__)" python -c "from src.command_parser import parse_command;
print('IMPORT PASS')" $env:PYTHONPATH = (Get-Location).Path
```

9. Development Environment

Component	Observed Configuration
Platform	Windows / PowerShell
Python	3.11.9
Virtual Environment	venv
pytest	9.1.1
Project Root	Project-03-AI-Smart-Device-Control
Source Package	src
Test Directory	tests

10. Troubleshooting / Lessons Learned

Issue	Resolution
pytest executed inside src/tests	Returned to the project root before running pytest.
No module named src	Created src/__init__.py and configured the package/import path.
command_model import error	Updated command_parser.py to import DeviceCommand through src.command_model.
Final collection	Project-root pytest collected all five tests successfully.

11. Security / Execution Boundary

The parser only creates structured commands. It does not execute arbitrary shell commands or hardware actions. This separation is intentional before laptop automation and device control are introduced.

12. Step 03 Milestone Status

Milestone	Status
Input normalization	COMPLETED
IoT parsing	COMPLETED
Laptop parsing	COMPLETED
Bluetooth parsing	COMPLETED
Unknown command handling	COMPLETED
Automated tests	5/5 PASS — 100%
Actual device execution	Pending — later steps

13. Project Status After Step 03

Step	Milestone	Status
01	Project Initialization & Environment Setup	COMPLETED
02	Project Configuration & Device Command Model	COMPLETED
03	Command Parser / Intent Model	COMPLETED — 5/5 PASS
04	Laptop Automation Foundation	Pending
05	Bluetooth Device Discovery	Pending
06	Bluetooth Device Control	Pending
07	IoT Control Foundation	Pending
08	AI Intent Integration	Pending
09	Voice Input Integration	Pending
10	Voice Response Integration	Pending
11	Command Router	Pending
12	Activity Logging	Pending
13	Error Handling	Pending
14	Dashboard Foundation	Pending
15	Dashboard Integration	Pending
16	Automated Testing	Pending — dedicated final suite later
17	End-to-End Testing	Pending
18	Performance Evaluation	Pending
19	Documentation & Final Report	Pending
20	Final Demonstration & Closure	Pending

14. Step 03 Conclusion

STEP 03 — COMMAND PARSER / INTENT MODEL: PASS. The project can now convert defined natural-language commands into structured DeviceCommand objects and safely represent unsupported commands. The final automated validation collected five tests and passed all five in 0.06 seconds.

15. Revision Record

Revision	Update
STEP 01	Project initialization, environment setup, validation, roadmap, and documentation foundation.
STEP 02	Configuration, configuration loader, DeviceCommand model, validation, and milestone status.
STEP 03	Deterministic command parser, supported intent mappings, unknown-command handling, package/import correction, and final pytest result of 5/5 PASS in 0.06s.
Next	Continue with Step 04 — Laptop Automation Foundation and update this same report after validation.